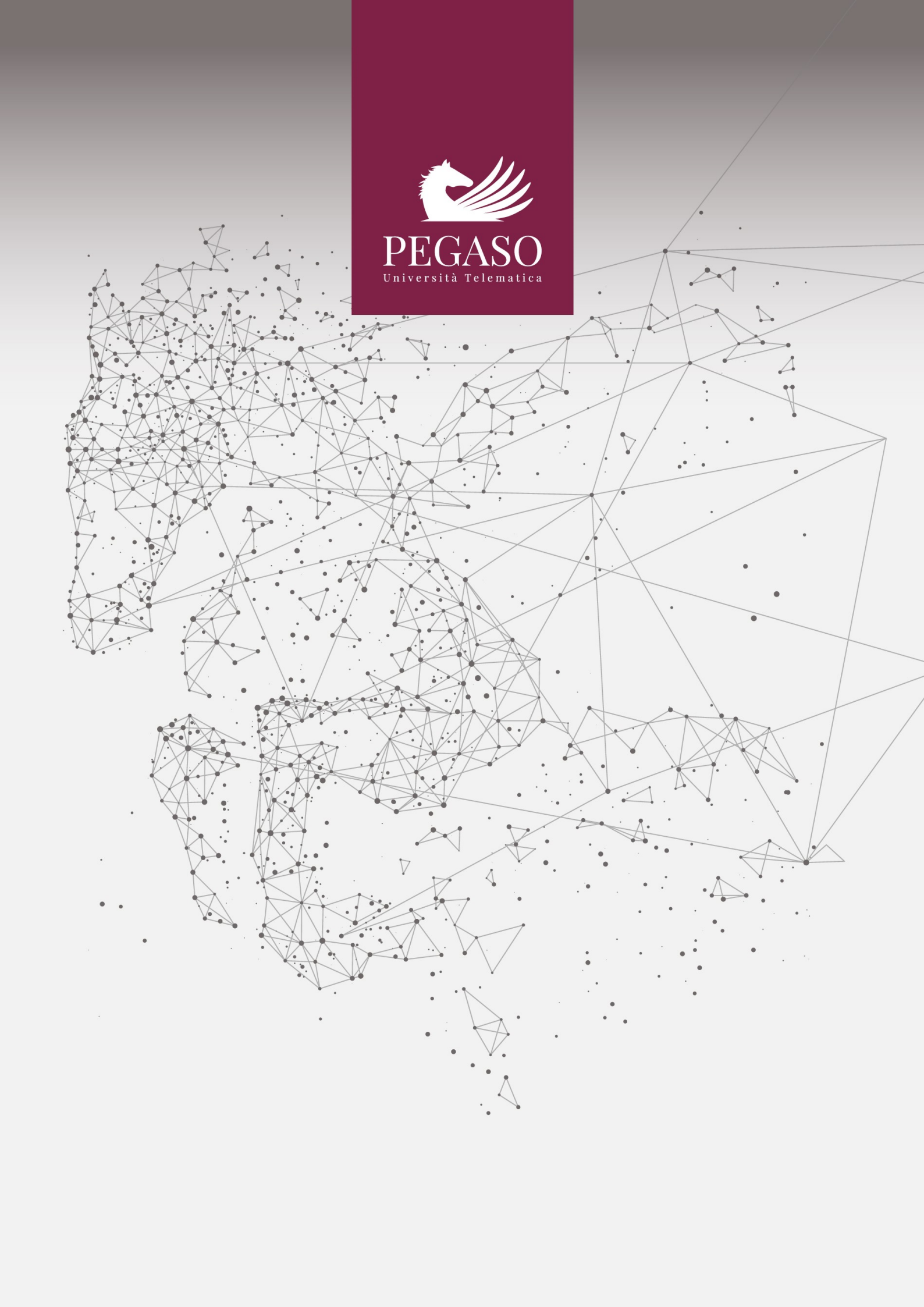

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. CODIFICA ASCII PER I CARATTERI 3
2. CODIFICA UNICODE PER I CARATTERI 8
3. GESTIONE DEI TIPI DI DATO CARATTERE E STRINGA IN ASSEMBLY MIPS 10
BIBLIOGRAFIA 15

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Codifica ASCII per i caratteri

Oltre alla rappresentazione di informazione numerica, è necessario disporre anche di opportune codifiche per l'elaborazione di testi alfanumerici.

Il termine **ASCII** (*American Standard Code for Information Interchange*, Codice Standard Americano per lo Scambio di Informazioni) identifica un codice standard per la rappresentazione di caratteri pubblicato dall'American National Standards Institute (ANSI) nel 1968. In italiano viene pronunciato *ascii* oppure *aschi*.

La codifica ASCII utilizza **sequenze binarie di 7 bit**, quindi può rappresentare un insieme di $2^7 = 128$ elementi. Ad ogni carattere è associata una sequenza binaria che in Notazione posizionale pesata rappresenta un valore compreso tra 0 e 127.

La prima delle seguenti tabelle mostra l'elenco dei codici dei caratteri stampabili. In essa sono inclusi 95 codici (da 32 a 126) relativi a caratteri stampabili costituiti da: le lettere maiuscole e minuscole dell'alfabeto inglese, le dieci cifre decimali, lo spazio bianco, caratteri di punteggiatura e alcuni simboli speciali come dollaro, parentesi, ecc.

La tabella mostra la sequenza binaria che rappresenta il simbolo, il corrispondente valore decimale ed il simbolo stampato.

Gli altri 33 codici (da 0 a 31 ed il 127) elencati nella tabella successiva sono relativi a simboli non stampabili utilizzati per la formattazione dei testi o come simboli di controllo. Ad esempio la sequenza di tutti 0 rappresenta il carattere NULL utilizzato nel Linguaggio C per indicare la fine di una stringa.

010 0000	32	**Spazio**	100 0000	64	@	110 0000	96	`
010 0001	33	**!**	100 0001	65	**A**	110 0001	97	**a**
010 0010	34	**"**	100 0010	66	**B**	110 0010	98	**b**
010 0011	35	#	100 0011	67	**C**	110 0011	99	**c**
010 0100	36	**$**	100 0100	68	**D**	110 0100	100	**d**

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

PEGASO
Università Telematica

010 0101	37	**%**	100 0101	69	**E**	110 0101	101	**e**
010 0110	38	**&**	100 0110	70	**F**	110 0110	102	**f**
010 0111	39	**'**	100 0111	71	**G**	110 0111	103	**g**
010 1000	40	**(**	100 1000	72	**H**	110 1000	104	**h**
010 1001	41	**)**	100 1001	73	**I**	110 1001	105	**i**
010 1010	42	*****	100 1010	74	**J**	110 1010	106	**j**
010 1011	43	**+**	100 1011	75	**K**	110 1011	107	**k**
010 1100	44	**,**	100 1100	76	**L**	110 1100	108	**l**
010 1101	45	**-**	100 1101	77	**M**	110 1101	109	**m**
010 1110	46	**.**	100 1110	78	**N**	110 1110	110	**n**
010 1111	47	**/**	100 1111	79	**O**	110 1111	111	**o**
011 0000	48	**0**	101 0000	80	**P**	111 0000	112	**p**
011 0001	49	**1**	101 0001	81	**Q**	111 0001	113	**q**
011 0010	50	**2**	101 0010	82	**R**	111 0010	114	**r**
011 0011	51	**3**	101 0011	83	**S**	111 0011	115	**s**

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

PEGASO
Università Telematica

011 0100	52	**4**	101 0100	84	**T**	111 0100	116	**t**
011 0101	53	**5**	101 0101	85	**U**	111 0101	117	**u**
011 0110	54	**6**	101 0110	86	**V**	111 0110	118	**v**
011 0111	55	**7**	101 0111	87	**W**	111 0111	119	**w**
011 1000	56	**8**	101 1000	88	**X**	111 1000	120	**x**
011 1001	57	**9**	101 1001	89	**Y**	111 1001	121	**y**
011 1010	58	**:**	101 1010	90	**Z**	111 1010	122	**z**
011 1011	59	**;**	101 1011	91	**[**	111 1011	123	**{**
011 1100	60	**<**	101 1100	92	****	111 1100	124	**|**
011 1101	61	**=**	101 1101	93	**]**	111 1101	125	**}**
011 1110	62	**>**	101 1110	94	**^**	111 1110	126	**~**
011 1111	63	**?**	101 1111	95	_			

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

CODICI ASCII NON STAMPABILI				
000 0000	0	NUL	**\0**	*Null* Carattere nullo
000 0001	1	SOH		
000 0010	2	STX		
000 0011	3	ETX		
000 0100	4	EOT		
000 0101	5	ENQ		
000 0110	6	ACK		
000 0111	7	BEL	**\a**	*Alert* segnale sonoro
000 1000	8	BS	**\b**	*Backspace* cancella all'indietro
000 1001	9	HT	**\t**	*Tab* tabulazione orizzontale
000 1010	10	LF	**\n**	*New line* nuova riga
000 1011	11	VT	**\v**	*Vertical tab* tabulazione verticale
000 1100	12	FF	**\f**	*Form feed* nuova pagina
000 1101	13	CR	**\r**	*Carriage return* ritorno a capo
000 1110	14	SO		
000 1111	15	SI		
001 0000	16	DLE		
001 0001	17	DC1		
001 0010	18	DC2		
001 0011	19	DC3		
001 0100	20	DC4		
001 0101	21	NAK		
001 0110	22	SYN		
001 0111	23	ETB		
001 1000	24	CAN		
001 1001	25	EM		
001 1010	26	SUB		
001 1011	27	ESC		
001 1100	28	FS		
001 1101	29	GS		
001 1110	30	RS		
001 1111	31	US		
111 1111	127	DEL		

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Un carattere non stampabile può essere rappresentato mediante una *sequenza di escape* costituita da una barra rovesciata seguita da una lettera associata al carattere, oppure dal numero che fornisce la codifica del carattere in notazione ottale o esadecimale.

La tabella dei caratteri non stampabili mostra la sequenza binaria che rappresenta il simbolo, il corrispondente valore decimale, l'abbreviazione della denominazione e la *sequenza di escape* dei simboli più utilizzati.

Per aumentare il numero di caratteri codificati, è stata definita un'estensione a 8 bit del codice ASCII, denominata *Extended ASCII*, che consente di avere $2^8 = 256$ caratteri rappresentabili con valori compresi tra 0 e 255.

Questa estensione include come sottoinsieme la codifica ASCII con 7 bit poiché i relativi codici vengono estesi a 8 bit aggiungendo come cifra più significativa un bit uguale a 0 (il valore rappresentato nella Notazione posizionale pesata non cambia).

Le ulteriori sequenze disponibili forniscono codici per altri simboli, in particolare per i caratteri con accenti o con segni diacritici che nei testi vengono aggiunti a una lettera per modificarne la pronuncia o per distinguere il significato di parole simili.

Tuttavia questo aumento non risulta sufficiente per far fronte alla crescente necessità di codificare i simboli degli alfabeti dei linguaggi scritti utilizzati nella comunicazione umana a livello mondiale, fortemente incentivata dallo sviluppo della comunicazione nel Web.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

PEGASO
Università Telematica

2. Codifica Unicode per i caratteri

Il codice ASCII anche nella sua forma estesa a 8 bit non consente di rappresentare i linguaggi utilizzati nella comunicazione mondiale, che comprendono gli alfabeti asiatici con migliaia di simboli ed ideogrammi.

Attualmente il sistema di codifica più utilizzato è quello aggiornato e diffuso dall'*Unicode Consortium*, un consorzio internazionale di aziende interessate alla interoperabilità nel trattamento informatico dei testi in lingue diverse.

Il sistema di codifica ***Unicode*** comprende diversi standard che prevedono l'uso di codifiche con sequenze di 8 bit, 16 bit e 32 bit denominati, rispettivamente, UTF-8, UTF-16 e UTF-32, dove UTF è l'acronimo di **U**nicode **T**ransformation **F**ormat.

Oltre alla rappresentazione dei simboli di quasi tutti i sistemi di scrittura attualmente usati a livello mondiale, nel sistema Unicode è disponibile anche la rappresentazione di caratteri relativi a molte lingue morte, di simboli matematici e musicali.

Il codice ASCII è compatibile con il sistema di codifica Unicode di cui costituisce un sottoinsieme, in quanto i primi 128 caratteri Unicode corrispondono al codice ASCII a 7 bit, ed i primi 256 al codice ASCII esteso a 8 bit.

I **Linguaggi C e C++** utilizzano lo **standard ASCII** per la rappresentazione dei caratteri di base.

Per adeguarsi alle nuove esigenze di rappresentazione dei testi, sono stati introdotti ulteriori tipi di carattere a dimensione fissa di 16 bit e 32 bit, detti *wide character* (caratteri ampi), per fornire la rappresentazione delle codifiche Unicode a 16 e 32 bit, e librerie standard forniscono una serie di funzioni per gestire i caratteri ampi e le stringhe che li includono.

Il **Linguaggio Java** utilizza lo standard ***Unicode a 16 bit*** **UTF-16** che rappresentava lo standard più diffuso al tempo della progettazione del linguaggio.

Poiché le 2^{16} sequenze binarie di tale lunghezza sono un sottoinsieme della codifica Unicode a 32 bit, è stato aggiunto al linguaggio Java uno schema di codifica per gestire i caratteri che non sono codificati da sequenze di 16 bit. Questi caratteri, detti *supplementary characters* (caratteri supplementari) sono codificati come coppie di caratteri, quindi utilizzando un totale di 32 bit.

Lo standard ***Unicode a 8 bit*** **UTF-8** usa gruppi di Byte in numero variabile per rappresentare i caratteri. Questo standard risulta particolarmente utile per il trasferimento tramite sistemi di posta elettronica a 8-bit, ed è divenuto rapidamente il più utilizzato per il Web.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

PEGASO
Università Telematica

Il vantaggio di usare la codifica con un numero di Byte variabile sta nel fatto che è possibile scrivere un testo in Italiano ed usare un numero di Byte di poco maggiore rispetto ad una codifica in ASCII esteso (solo le lettere accentate occuperanno 2 Byte), mentre un testo in lingua Cinese avrà ogni carattere di tre Byte, con numeri, lettere, punteggiatura ed altri eventuali caratteri della codifica ASCII presenti nel testo di un solo Byte.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Gestione dei tipi di dato carattere e stringa in Assembly MIPS

I Linguaggi C, C++ e Java dispongono del **Tipo di dato CARATTERE** definito dalla dichiarazione *char* per la gestione dei caratteri di base

I Linguaggi C e C++ utilizzano la codifica ASCII e rappresentano i caratteri mediante interi di 8 bit che, in assenza di ulteriori specificazioni, assumono i valori default compresi nell'intervallo [−128, 127]. Si può anche dichiarare esplicitamente questo intervallo di variabilità mediante la dichiarazione *signed char.*

Usando la dichiarazione *unsigned char* la sequenza di 8 bit viene considerata un intero *unsigned* con valori nell'intervallo [0, 255] ed è possibile rappresentare la codifica ASCII estesa.

Il Linguaggio Java utilizza la codifica Unicode a 16 bit con valori nell'intervallo $[0, 2^{16}-1] = [0, 65535]$

Nelle istruzioni una costante di Tipo carattere viene rappresentata includendo tra singoli apici il relativo simbolo stampabile (esempio 'A').

Un carattere non stampabile può essere rappresentato mediante la relativa *sequenza di escape* costituita da una barra rovesciata seguita da una lettera associata al carattere (esempio '\b' rappresenta il simbolo *Backspace*), oppure dal numero che rappresenta la codifica del carattere in notazione ottale o esadecimale (esempio il simbolo NULL è rappresentato da \000 in ottale e da \x00 in esadecimale). Inoltre, la rappresentazione mediante la *sequenza di escape* viene anche utilizzata per i caratteri che hanno un significato riservato, come ad esempio il simbolo di barra rovesciata '\\' che indica l'inizio della *sequenza di escape,* o l'apice singolo '\' ' che viene usato per rappresentare una costante carattere.

Una *sequenza di escape* può essere usata all'interno degli apici singoli per rappresentare sia caratteri non stampabili o riservati, sia caratteri stampabili.

Per i caratteri stampabili le due rappresentazioni sono equivalenti ed intercambiabili, per cui si ha che '\062' equivale a '2' poiché il codice ottale 062 (in decimale 50) corrisponde al carattere 2.

Sia i codici ASCII a 7 bit e ad 8 bit, utilizzati nei Linguaggi C e C++, sia i codici Unicode a 16 bit usati in Java, possono essere indicati come **operandi immediati nelle istruzioni Assembly MIPS**, e scritti nei 16 bit del *campo immediato* della corrispondente istruzione in Linguaggio Macchina.

Per la gestione dei caratteri vengono prevalentemente usate le istruzioni Logiche poiché hanno operandi *unsigned.*

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

PEGASO
Università Telematica

ESEMPIO: L'istruzione di assegnamento:

```
char  val;
val = 'A'  ;
```

con l'assegnazione:

val ⟷ $t0

può essere tradotta dall'istruzione Assembly MIPS:

```
ori  $t0,  $zero,  65
```

Il codice ASCII che rappresenta il carattere 'A' è il numero 65, in binario 1000001, e viene scritto nei 16 bit dell'*operando immediato* dell'istruzione in Linguaggio Macchina che traduce *ori* mediante la sequenza:

0000000001000001

Le operazioni Logiche sono eseguite dall'Unità Aritmetico Logica (ALU) che richiede operandi di 32 bit. Poiché le istruzioni Logiche hanno operandi *unsigned,* si aggiungono 16 bit più significativi tutti uguali a 0 alla sequenza binaria dei 16 bit che rappresenta il valore 65, ottenendo così l'operando costante di 32 bit dell'istruzione *ori* (che rappresenta sempre il valore 65):

00000000000000000000000001000001

L'esecuzione di *ori* effettua l'OR *bit a bit* tra questo operando costante e l'operando con valore 0 contenuto nel registro $zero, e produce come risultato una sequenza uguale a quella dell'operando costante 65, che viene scritta nel registro destinazione $t0 realizzando l'assegnamento val = 'A' :

```
contenuto di $zero    00000000000000000000000000000000   OR
operando costante     00000000000000000000000001000001   =
      65  ⟷          00000000000000000000000001000001
```

Alcune caratteristiche della codifica ASCII rendono semplici le operazioni di base di elaborazione dei testi.

Ad esempio, nella tabella della codifica ASCII si può notare che i codici delle lettere minuscole e maiuscole crescono in modo continuo seguendo l'ordinamento alfabetico, e i caratteri delle cifre decimali crescono seguendo l'ordine di grandezza.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

PEGASO
Università Telematica

Poiché i codici sono numeri interi, è possibile utilizzare le **relazioni d'ordine tra numeri interi** per gestire l'ordinamento alfabetico tra caratteri.

ESEMPIO

Le seguenti istruzioni in Linguaggio C stabiliscono l'ordinamento alfabetico tra due caratteri:

```
char  primo = 'S',  secondo = 'B',  temp,  min  ;
if (secondo <  primo  ) {
        temp = primo   ;
        primo = secondo  ;
        secondo = temp  ; }
min = primo  ;
```

assegnando alle variabili i registri con i seguenti indirizzi:

primo	secondo	temp	min
$s1	$s2	$t0	$s5

una traduzione in Assembly MIPS è data da:

```
slt  $t3, $s2, $s1              # pone il contenuto di $t3=1 se
   #  secondo < primo  è  vera, altrimenti lo pone uguale a 0
beq $t3, $zero, END_IF      #       salta a END_IF
            #  se  contenuto di $t5 == 0  è  vera,  ⟺
            #  secondo < primo è  falsa
addi $t0, $s1, 0                #   temp = primo
addi $s1, $s2, 0                #   primo = secondo
addi $s2, $t0, 0                #   secondo = temp
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
END_IF   add  $s5,  $zero,  $s1          #   min = primo
```

Una ulteriore proprietà della codifica ASCII, che consente l'uso di **operatori aritmetici** per l'elaborazione dei testi, è relativa ai codici dei caratteri alfabetici maiuscolo e minuscolo che differiscono di un valore costante pari a 32. Le proprietà possono essere sfruttate sia nei Linguaggi ad Alto livello che nella traduzione in Assembly.

ESEMPIO:

carattere		codifica ASCII decimale
A	⟷	65
a	⟷	97 = 65 + 32

Le seguenti istruzioni in Linguaggio C cambiano da maiuscolo in minuscolo utilizzando l'operatore aritmetico + per la modifica:

```
if (carattere ==  'A'  )  {
      carattere = carattere + 32   ;  }
carattere = 'B'  ;
```

assegnando alla variabile carattere il registro con indirizzo $s1, una traduzione in Assembly MIPS è data da:

```
        subi  $t0, $s1, 65
        bne $t0, $zero, END_IF      #    salta a END_IF
            #  se la sottrazione ha dato un risultato diverso da 0,   #
        carattere⟺'A'  è  falsa
        addi $s1, $s1, 32           #   carattere = carattere + 32
END_IF  ori  $s1,  $zero,  66       #   carattere = 'B'
```

Il Tipo di dato **STRINGA** è una struttura dati costituita da una sequenza di caratteri e da operatori che effettuano elaborazioni come la concatenazione di stringhe ed il confronto.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

PEGASO
Università Telematica

Nei Linguaggi C, C++ e Java una costante di Tipo *stringa* è rappresentata racchiudendola tra doppi apici, ad esempio:

"Testo della stringa".

Un carattere all'interno di una stringa racchiusa tra doppi apici può essere rappresentato anche utilizzando la corrispondente *sequenza di escape*, sia nel caso di caratteri stampabili sia nel caso di caratteri non stampabili o riservati, ad esempio:

"Testo di un\'altra stringa".

Il Linguaggio C implementa il Tipo di dato *stringa* come un **Array di caratteri** il cui ultimo elemento è il carattere NULL che ha codice 0 (in binario 0000000).

La differente rappresentazione del carattere 0 che ha codice 48 (in binario 0110000) consente di avere all'interno della *stringa* anche caratteri 0 senza confusione con il carattere di fine *stringa* NULL.

Il Linguaggio Java implementa il Tipo di dato *stringa* con la **classe *String***. Quindi le stringhe di caratteri sono oggetti e gli operatori sono metodi della classe.

Il linguaggio C++ supporta entrambe le implementazioni.

La gestione del Tipo di dato *stringa* in **Assembly** si ottiene mediante un **Array di caratteri**.

La rappresentazione in Memoria di un carattere richiede una locazione (Byte) nella codifica ASCII, due locazioni (mezza parola) o quattro locazioni (parola) nelle codifiche Unicode a 16 bit e a 32 bit.

In Assembly MIPS sono disponibili istruzioni di trasferimento dati per tali lunghezze.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- D. A. Patterson, J. L. Hennessy, "Struttura e progetto dei calcolatori", Zanichelli.
- ORACLE Java Documentation, The Java Tutorials, Primitive Data Types
  https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html
- ORACLE Java Documentation, The Java Tutorials, Unicode
  https://docs.oracle.com/javase/tutorial/i18n/text/unicode.html

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

PEGASO
Università Telematica